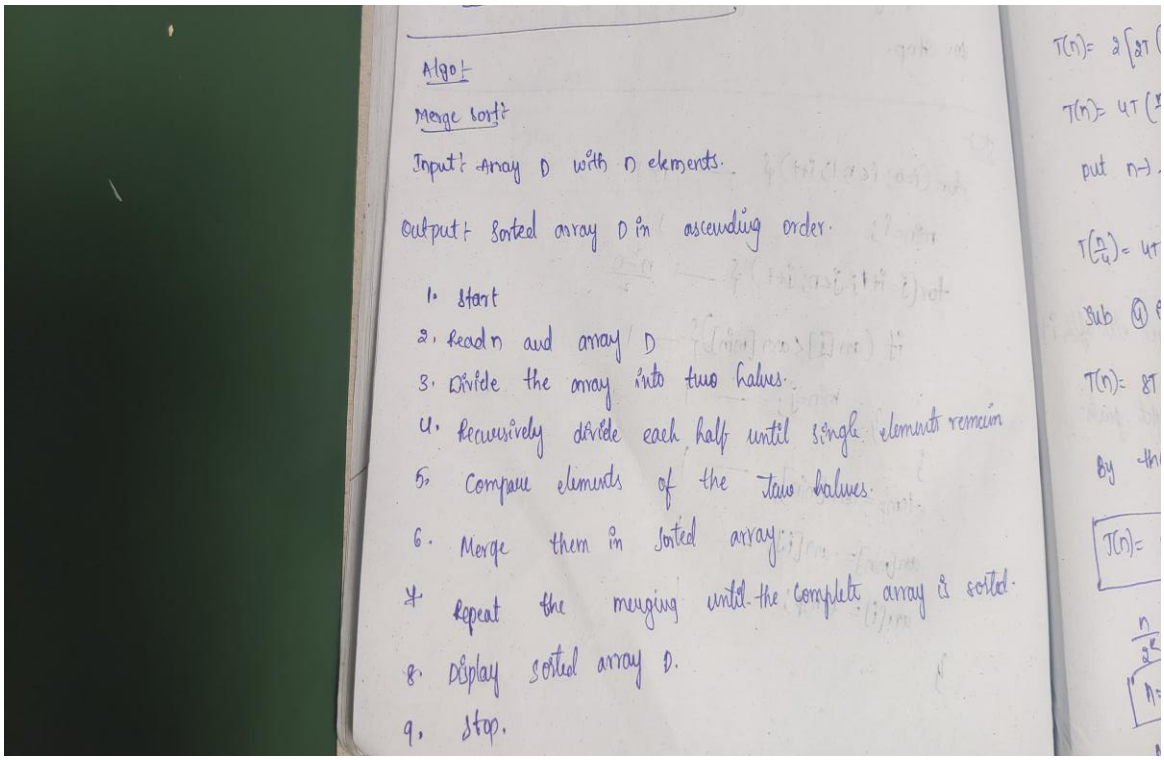
Aim: To Implementation and time analysis of Merge sort algorithm.

Algorithm for Merge sort:



Program of Merge sort:

```
#include <iostream>

using namespace std;


class MergeSortApp

{

public:

   int arr[100], n;
```

```
void arrayInput()
{
    cout << "Enter number of
elements: ";
    cin >> n;

    cout << "Enter the elements: ";
    for (int i = 0; i < n; i++)
    {
        cin >> arr[i];
    }
}

void printArray()
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}
```

```
void merge(int low, int mid, int high)
{
   int temp[100];
   int i = low;
   int j = mid + 1;
   int k = low;
   while (i <= mid && j <= high)
   {
      if (arr[i] <= arr[j])
      {
         temp[k] = arr[i];
         i++;
      }
      else
      {
         temp[k] = arr[j];
         j++;
      }
      k++;
   }

   while (i <= mid)
```

```
    {
        temp[k] = arr[i];
        i++;
        k++;
    }
    while (j <= high)
    {
        temp[k] = arr[j];
        j++;
        k++;
    }
    for (int x = low; x <= high; x++)
    {
        arr[x] = temp[x];
    }
}
void mergeSort(int low, int high)
{
    if (low < high)
    {
        int mid = low + (high - low) / 2;
        mergeSort(low, mid);
```

```
            mergeSort(mid + 1, high);
            merge(low, mid, high);
        }
    }
    // Merge Sort Function
    void sort()
    {
        mergeSort(0, n - 1);
    }
};
int main()
{
    MergeSortApp ms;
    ms.arrayInput();
    cout << "\nArray before sorting: ";
    ms.printArray();
    ms.sort();
    cout << "\nArray after Merge Sort: ";
    ms.printArray();
    return 0;
}
```

Output:

```
Output

Enter number of elements: 5
Enter the elements: 2
1
8
4
3

Array before sorting: 2 1 8 4 3

Array after Selection Sort: 1 2 3 4 8


=== Code Execution Successful ===
```

Time complexity: